COS 214 Practical 3: EventFlow

A Live Event Coordination Engine

Team members:

Chitambala Sikazwe (u25173384)

Gary Mull (u25071247)

Louwrens Johansen (u25607414)

1. Event Concept: City Lights Festival

City Lights Festival is a single-day, multi-stage outdoor music and community festival held on a large open showground. The festival is deliberately large and physically spread out. Attendees move between a main performance area and a river-front leisure area, and each of these behaves as a semi-independent operational zone while still reporting into one central event. This physical spread, combined with the fact that conditions on the ground (weather, capacity, schedule, safety) change throughout the day and need to reach the right operational units quickly, is the situation EventFlow is built to coordinate: a genuine part-whole containment tree (Composite) layered with a genuine one-to-many notification network (Observer).

The showground is organised as a root Venue containing two Zones, one of which itself contains a nested Precinct. This gives the tree the three levels of Composite nesting required below the root. Concretely:

- Venue (root): City Lights Festival, the whole showground for the day.
- Zone: Main Stage Zone, the headline performance area.
- Zone: River Side Zone, a leisure zone along the river that contains a nested Precinct.
- Precinct (nested inside River Side Zone): Food Court Precinct, a cluster of vendors treated as one sub-group.

Venue, Zone and Precinct are three conceptually different kinds of grouping/area, all implemented by the single CascadingZone class, a Composite group that also plays the Observer and Subject roles (see section 3). This is the point of Composite: the same class recurses to any depth without the client needing a different type per level.

Within these groups sit five concrete kinds of operational unit (Leaves), each with meaningfully different behaviour and not just a different printed name:

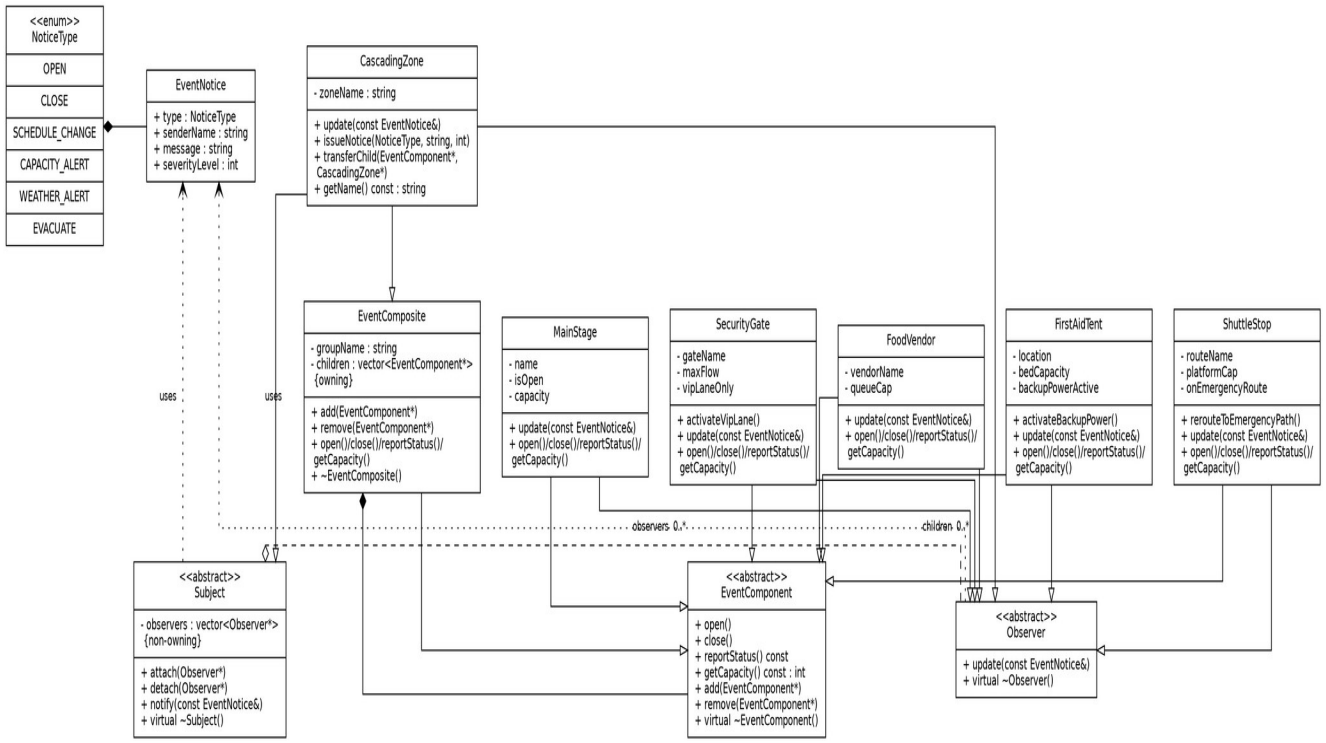
- MainStage: hosts performances, pauses on a weather alert or evacuation, resumes on an all-clear.
- SecurityGate: controls attendee admission. On a capacity alert it checks its own severity threshold and either restricts admission to a single VIP fast-track lane or closes fully.
- FoodVendor: serves food from a stall, and suspends kitchen service on a weather alert or evacuation.
- FirstAidTent: provides first aid, and deliberately stays active through most alerts. On a severe weather alert it switches to backup power rather than closing.
- ShuttleStop: runs a transport route between zones, and reroutes to an emergency path on a weather alert or evacuation rather than stopping.

A notice originates directly from whichever CascadingZone the incident is local to (or from the root Venue, for a festival-wide notice), through issueNotice(NoticeType, message, severity). EventFlow does not introduce a separate coordinator class for this: CascadingZone already owns everything

issuing a notice requires, since it is a Subject in its own right, so wrapping that single responsibility in another class would only duplicate it. A notice cascades down through whichever zones and units registered interest, reaching only those that actually need to know.

2. Completed UML Class Diagram

The diagram below completes the architecture that Figure 1 of the brief left open, using the class names actually implemented in the codebase. EventComponent and the five leaf classes belong to the Composite side, Subject and Observer belong to the Observer side, CascadingZone sits at the intersection of both since it inherits from EventComposite, Subject and Observer at once, and EventNotice/NoticeType is the shared data the two collaborations pass around. Solid diamonds are ownership (composition), hollow diamonds and dashed lines are non-owning association/aggregation, and open triangles are inheritance.



Multiplicities: one EventComposite owns 0..* EventComponent children (composition, filled diamond), one Subject tracks 0..* Observer registrations without owning them (aggregation, hollow diamond). Every abstract base with a virtual member (EventComponent, Subject, Observer) declares a virtual destructor, and CascadingZone, which inherits from EventComposite, Subject and Observer at once, correctly runs all three destructor chains when a subtree is deleted.

3. GoF Participant Mapping (Observer and Composite)

The table below lists every GoF role required by the practical and states which EventFlow class fills it. CascadingZone appears more than once because it takes part in more than one collaboration. Each row explains the specific collaboration that role belongs to so no relationship is double counted.

Pattern	GoF Role	EventFlow Class(es)	Why it plays this role
Composite	Component	EventComponent	Declares open(), close(), reportStatus(), getCapacity() and default (throwing) add()/remove() so the client can treat leaves and

Pattern	GoF Role	EventFlow Class(es)	Why it plays this role
			groups the same way.
Composite	Leaf	MainStage, SecurityGate, FoodVendor, FirstAidTent, ShuttleStop	Concrete operational units with no children. Each one implements the four common operations, and update(), with genuinely different behaviour.
Composite	Composite	EventComposite (base) and CascadingZone (the concrete zones actually used in the tree)	EventComposite owns a vector of EventComponent* and forwards open()/close()/reportStatus()/getCapacity() recursively to its children. CascadingZone extends it with the Observer/Subject roles below.
Composite	Client	main.cpp / Task8Demo.cpp	Builds the tree and calls operations through the EventComponent interface without knowing which concrete type it is holding.
Observer	Subject	Subject	Declares attach(), detach() and notify(const EventNotice&), and owns the (non-owning) list of registered observers.
Observer	ConcreteSubject	CascadingZone	Any CascadingZone can originate a notice directly, through issueNotice(), and every CascadingZone relays a notice it receives to its own registered observers.
Observer	Observer	Observer	Declares update(const EventNotice&), the single entry point every interested party has to implement.
Observer	ConcreteObserver	CascadingZone (relative to its parent) and every concrete leaf	A CascadingZone registers with the zone above it. Every leaf registers with the CascadingZone that contains it, and reacts to the same notice in its own way.

CascadingZone is the class worth pausing on. Within the Composite collaboration it behaves purely as a Composite (it inherits EventComposite's ownership and recursion). Within the Observer collaboration it plays two separate roles depending on which edge of the tree is being discussed: ConcreteObserver when receiving a notice from whatever is above it, and ConcreteSubject when originating a notice of its own or re-broadcasting one to whichever of its own children registered for it. These are two different collaborations between two different pairs of objects, not one pattern doing double duty (expanded on in section 4(e)).

4. Design Rationale

(a) Why Composite is a genuine part-whole tree. A SecurityGate or a FoodVendor has no meaningful existence outside the Zone or Precinct that physically contains it on the showground. It is not just associated with its group, it is part of it, and deleting the group must delete everything inside it. EventComposite expresses this with real ownership: children are stored as pointers whose lifetime it controls, and ~EventComposite() deletes every child exactly once via the virtual ~EventComponent() destructor chain. The four common operations (open, close, reportStatus, getCapacity) are also whole-part operations by nature, since a Zone's capacity genuinely is the sum of its parts' capacities, which is the strongest sign that Composite, and not a flat list with manual iteration, is the right structure.

(b) Why Observer is required rather than direct hard-coded calls. No single object can know at compile time which units exist in a given festival build, or which of them care about a WEATHER_ALERT versus a SCHEDULE_CHANGE. FirstAidTent stays active through an evacuation but reacts to a severe weather alert by switching to backup power, while ShuttleStop reroutes instead of closing. Hard-coding something like "if MainStage then pause, if FoodVendor then suspend" inside a single coordinating object would break the practical's own rule against type-checking dispatch, and would force that object to be recompiled every time a new unit type is added. Observer decouples "a notice happened" from "who cares and how they react": units attach and detach at runtime, and a CascadingZone's own notify() loop never needs to change no matter what is registered with it.

(c) Who owns event components. Ownership of the Composite tree is single and exclusive: whichever EventComposite (in practice, a CascadingZone) currently holds a child in its children vector owns it, and only that zone may delete it. When a unit is transferred between zones, CascadingZone::transferChild() calls the old parent's remove() to completion first, erasing the pointer from its vector without deleting the object, before the new parent's add() takes the same pointer. Ownership moves in one step from one zone to the other and is never shared, which is what stops a double free.

(d) Who owns, or does not own, observer references. Subject stores Observer* as plain, non-owning pointers. Attaching a unit as an observer is registering interest, not taking responsibility for its lifetime. EventFlow's Subject deliberately does not attempt to track that lifetime automatically: attach() and detach() only ever manage the registration list itself, and Observer's destructor does not try to self-detach from whatever it was registered with. Instead, the responsibility is placed with whichever piece of client code already has the most context at the moment it matters, for example CascadingZone::transferChild(), which detaches an object from its old zone's Subject side and re-attaches it to the new zone's Subject side in the same operation that moves its Composite ownership, so the two registrations never drift apart. Section 4(f) below sets out this attach/detach registration policy in full.

(e) Why a class that is both Subject and Observer is not a misuse of either pattern. GoF patterns describe a collaboration between roles, not a permanent label on a class. CascadingZone's Observer role exists in its collaboration with whatever sits above it (it is being told something), and its Subject role exists in a completely separate collaboration with whatever sits below it (it is telling others something) or, when it originates a notice itself, with the units that registered directly with it. These are two different one-to-many relationships with two different sets of participants, and CascadingZone never plays both roles on the same edge of the tree at once, since a notice always flows strictly downward through a chain of otherwise normal Observer/Subject pairs. The dual role is exactly what the brief's own scenario asks for, that "some event areas receive notifications from above and then notify interested units below."

(f) Registration policy (attach/detach)

Policy for attach(Observer *observer): ignores duplicate registrations, so if the observer is already in the vector it will not be added again.

Policy for detach(Observer *observer): if the observer is found in the vector it is removed; if it is not found, the call does nothing.

The Subject does not own its observers, and observers do not detach themselves automatically. The client is made responsible for the lifetime of the observer: Subject stores non-owning pointers to observer objects and cannot delete them, attach()/detach() are used purely for runtime registration and carry no ownership meaning, and an observer cannot detach itself automatically, this has to be done by the client. Placing that responsibility with the client, rather than trying to track it automatically inside Subject or Observer, means the client is always the one that decides when an object is detached and when it is destroyed, which keeps that ordering explicit and easy to reason about even though CascadingZone's cascading structure is not trivial.

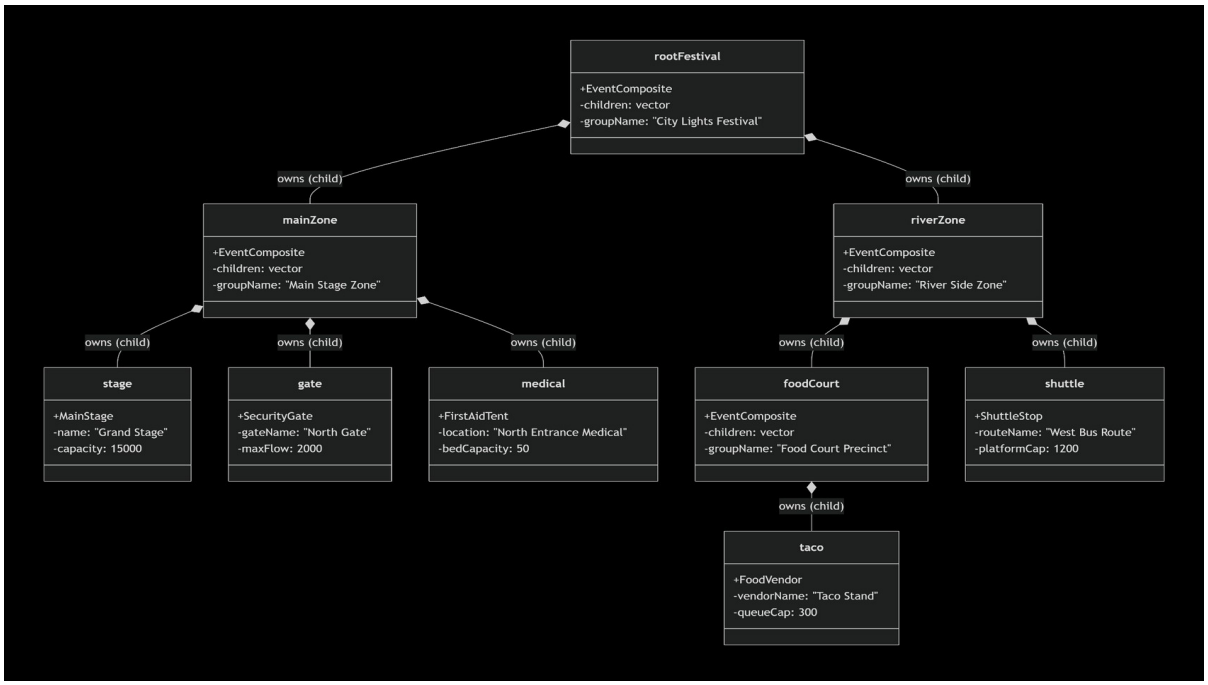
(g) Push versus pull

EventFlow uses a push implementation: Subject::notify(const EventNotice&) takes an EventNotice and passes it directly to each observer's update() method.

Advantage: the push model is simple and easy to adjust. The Subject pushes data to the observer as part of the notification itself, so the observer never has to query the Subject back for state, which keeps the Observer implementation simpler and makes the interaction between objects more direct.

Trade-off: coupling. The Subject and Observer are coupled through the shape of the EventNotice object, so if that format changes, every observer that reads it may need to be updated too. This makes the push model less flexible than a pull model in that one respect, which was judged an acceptable trade-off for the simplicity it buys elsewhere.

5. Composite Object Diagram



The complete owned subtree is released exactly once due to the ownership policy implemented in the EventComposite class. When the root object is deleted, the destructor is called and the destructor iterates through its children vector and deletes all the child objects. Due to every object being an EventComposite, it recursively deletes all its children.

Ownership rules for transferring an object between composites: the owner is a parent, it gets detached from the old parent with remove() then attached onto the new parent with add(). Since we used the remove and add functions there will not be any double deletion.

6. Event Rules and Original Features

This section builds directly on the Observer classes described above: EventNotice, Observer, Subject, and CascadingZone. The five concrete leaves were extended to also implement Observer, and CascadingZone gained one method, transferChild(), for the runtime reorganisation described below.

6.1 Five event rules

Each of the five leaf classes implements update(const EventNotice&) and reacts differently to the same notice, through polymorphism rather than a type check in a controller. The clearest example is a WEATHER_ALERT:

- MainStage: pauses the performance (close()). An outdoor stage cannot safely keep running.
- SecurityGate: has no rule for WEATHER_ALERT specifically, only for CAPACITY_ALERT and EVACUATE, so it logs that it has no reaction and stays as it was. This is itself a deliberate difference: not every unit treats every notice as relevant.
- FoodVendor: suspends kitchen service (close()).
- FirstAidTent (MedicalTeam): stays active. If the notice's severityLevel is 4 or higher it also switches to backup power, but it never closes.
- ShuttleStop: reroutes to an emergency path and keeps running, rather than stopping.

This is visible directly in the demonstration output (Task4Testing.cpp, testTask4()): a single call to rootFestival->issueNotice(WEATHER_ALERT, ...) cascades through Main Stage Zone and River Side Zone and results in the stage stopping, the medical tent switching to backup power, the vendor halting service and the shuttle rerouting, all from the one notice. Because Food Court Precinct is itself a CascadingZone attached to River Side Zone, the same call also shows the notice reaching a leaf (the Taco Stand) three runtime levels below the root, which is the same cascading behaviour required elsewhere and confirms the two mechanisms are working from the same notification chain.

6.2 Runtime reorganisation

CascadingZone::transferChild(EventComponent* child, CascadingZone* newParent) moves the Taco Stand vendor from the Food Court Precinct to the Main Stage Zone while the program is running. It updates two things that must both change together:

- Composite ownership: remove(child), inherited from EventComposite, takes the pointer out of the old zone's children vector without deleting the object, then newParent->add(child) gives ownership to the new zone.
- Observer registration: if the child is also an Observer, detach(childObserver), inherited from Subject, removes it from this zone's observer list (safely doing nothing if it was not registered there), and newParent->attach(childObserver) registers it with the new zone.

The demonstration shows this is not just cosmetic. After the transfer, the Taco Stand appears in Main Stage Zone's status report instead of Food Court Precinct's, and when a later OPEN notice is sent, the Taco Stand reopens as part of Main Stage Zone's cascade, not Food Court Precinct's, which confirms the observer registration actually moved and not only the ownership pointer.

6.3 A condition-based decision

SecurityGate::update() contains a condition ready to become an alt fragment with guards:

```
if (notice.severityLevel >= closeThreshold) { close(); }
else { activateVipLane(); }
```

A CAPACITY_ALERT with a severityLevel below the gate's own closeThreshold (4) only restricts admission to a single VIP lane (see 6.4 below). A severityLevel at or above the threshold closes the gate completely.

6.4 Three original features

1. SecurityGate VIP fast-track lane. Instead of a binary open/closed gate, a moderate CAPACITY_ALERT restricts the gate to a single fast-track lane (activateVipLane()) rather than shutting it completely, which is closer to how a real festival gate would behave under pressure. The logic lives entirely inside SecurityGate: it only adds one boolean flag, one threshold constant and one private method to a class that already owned this decision, so it does not turn SecurityGate or anything else into a god object.

2. MedicalTeam backup power. A severe WEATHER_ALERT (severityLevel 4 or higher) makes FirstAidTent switch to backup power (activateBackupPower()) instead of doing nothing or closing, reflecting that a safety-critical unit needs a stronger response than an ordinary one.

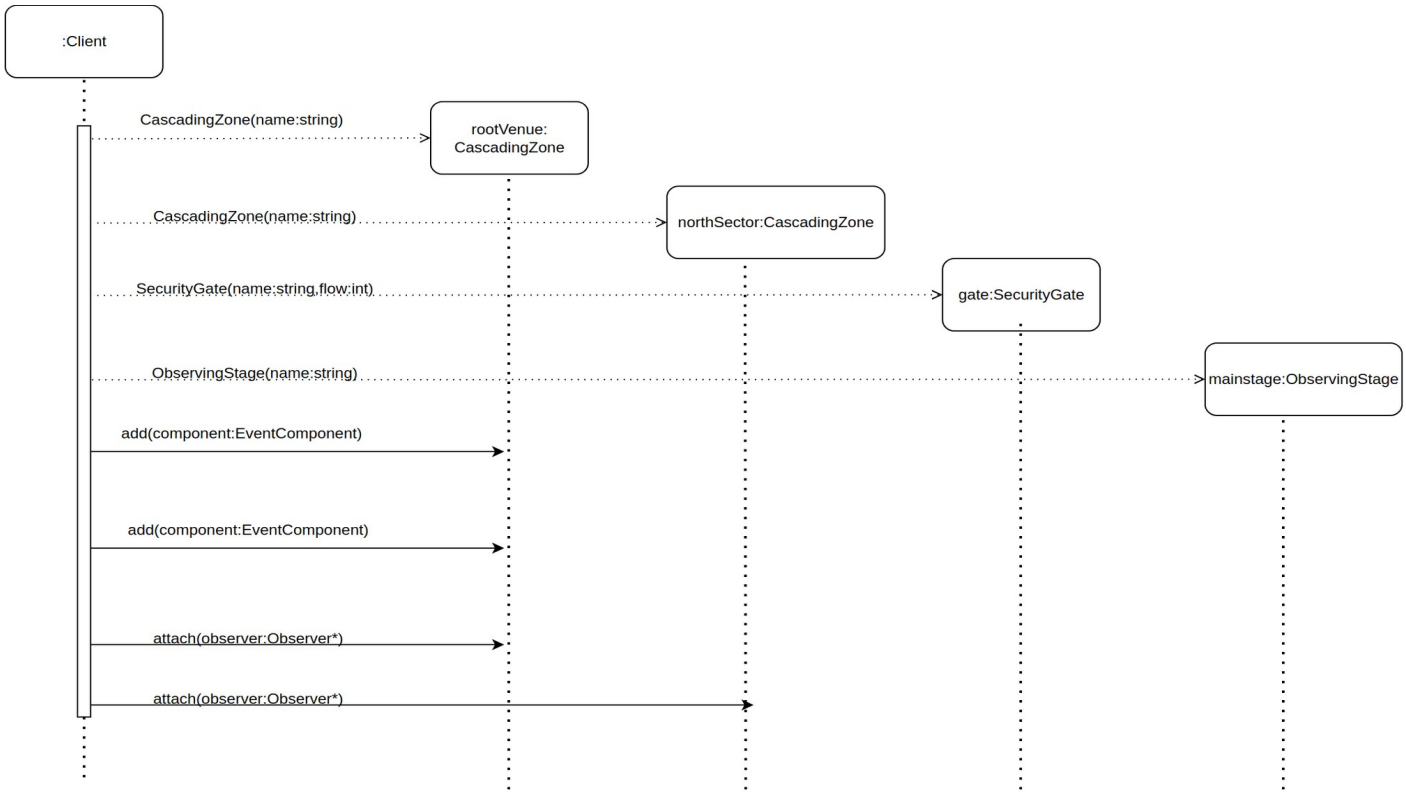
3. ShuttleStop dynamic rerouting. Rather than simply stopping like the other units, ShuttleStop reacts to WEATHER_ALERT and EVACUATE by rerouting itself onto an emergency path (rerouteToEmergencyPath()) and continuing to run, since transport is one of the few things attendees need more of during an incident, not less.

All three features are contained inside the one leaf class they belong to. Neither EventComposite nor CascadingZone ever inspects a concrete leaf type or branches on which feature a unit supports; they only ever call open(), close(), reportStatus(), getCapacity() or update() through the EventComponent/Observer interfaces, so adding these features did not require touching any other part of the codebase.

The NoticeType enum covers OPEN, CLOSE, SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT and EVACUATE, six types in total, which satisfies the practical's minimum of six without a separate PAUSE/RESUME pair. The event rules above use OPEN as the signal that restores normal operation (reopening a stage, gate, vendor or shuttle, and switching the medical tent off backup power), which fits naturally alongside CLOSE and EVACUATE.

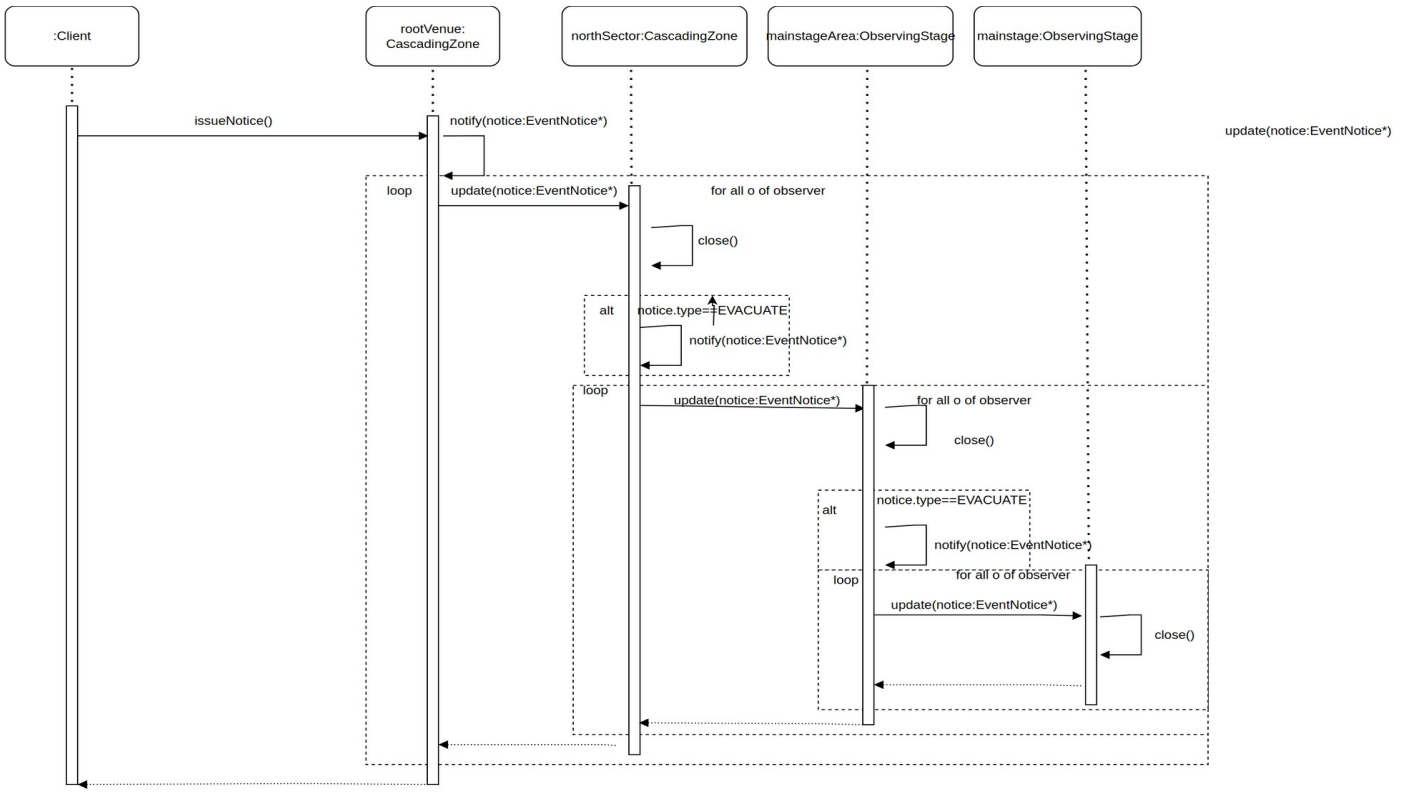
7. SD1: Building and Registering Part of the Event

Shows the client creating a root Composite, a nested Composite, and two Leaves, adding them to the ownership tree, and establishing the Observer registrations needed for later notifications.



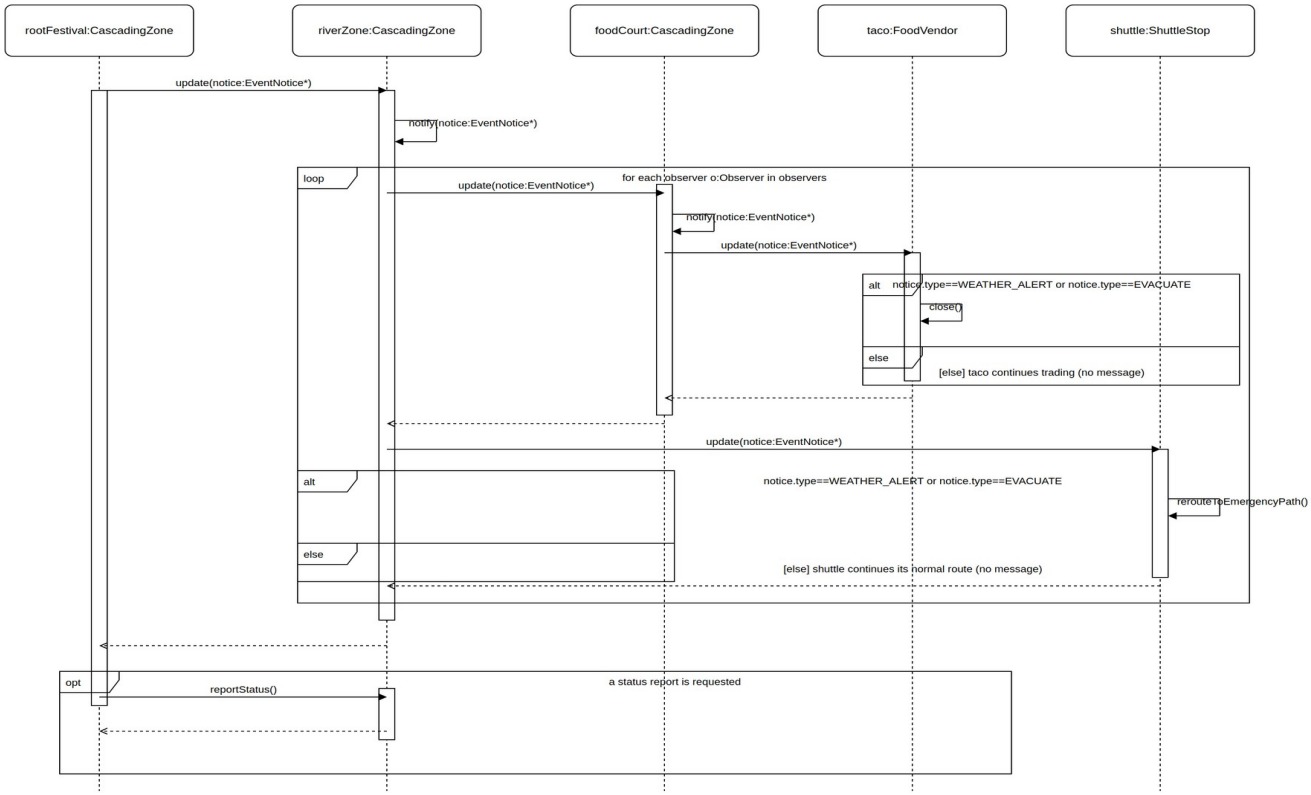
8. SD2: Cascading Event Notification

Shows a notice originating from a Subject and cascading through three runtime levels to multiple observers, using a loop fragment where observers are notified and an alt fragment for an EVACUATE-triggered close().



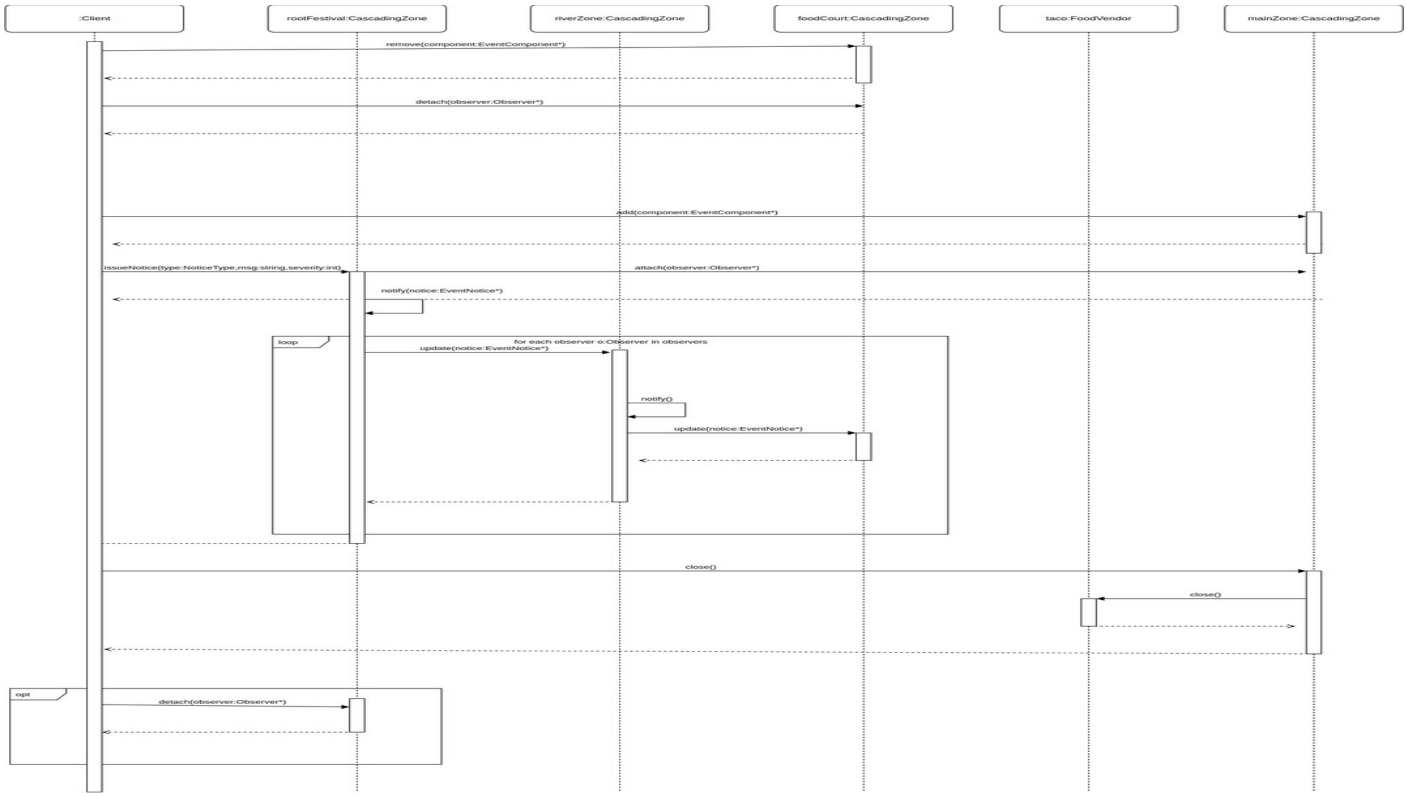
9. SD3: Conditional Event Response and Composite Behaviour

Models a weather scenario at River Side Zone: the zone relays the notice to its registered observers through its own notify() cascade (a real Composite/Observer operation, not a fabricated one), and two alt fragments show FoodVendor and ShuttleStop each deciding their own reaction through their own update() override, exactly matching the switch-statement logic in the code. An opt fragment shows an optional reportStatus() query, a genuine Composite operation, issued after the cascade completes.



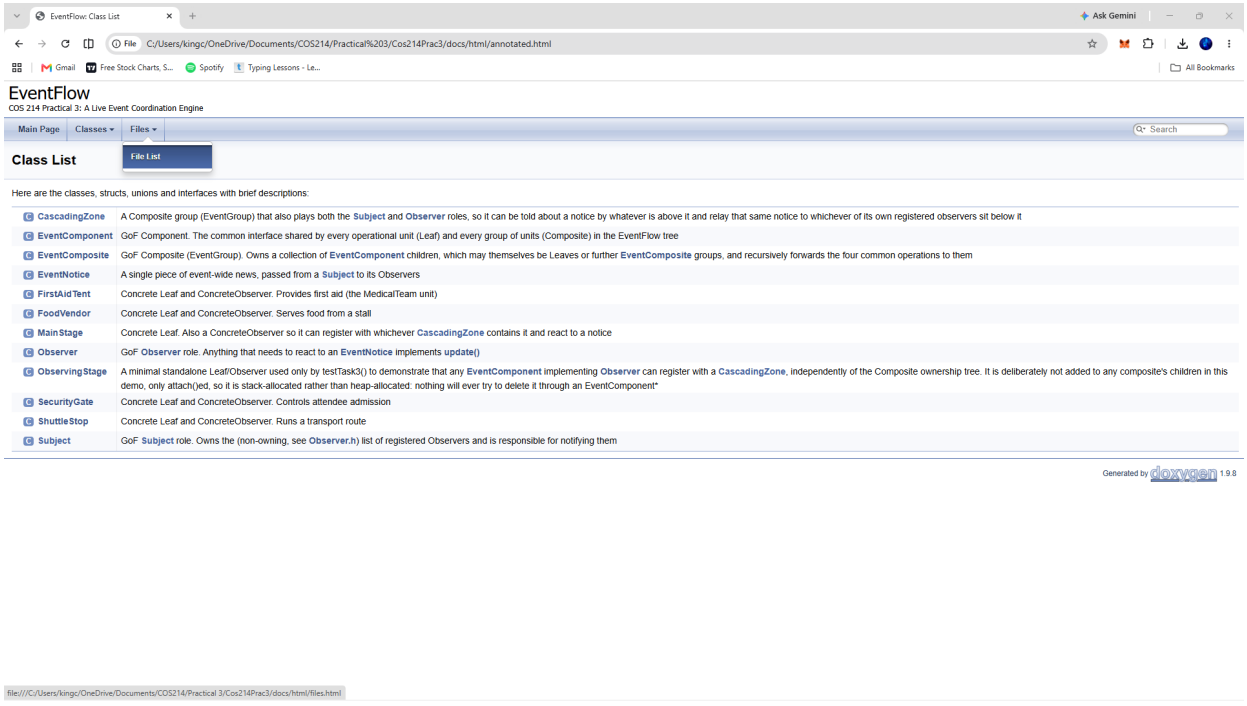
10. SD4: Signature Event Scenario

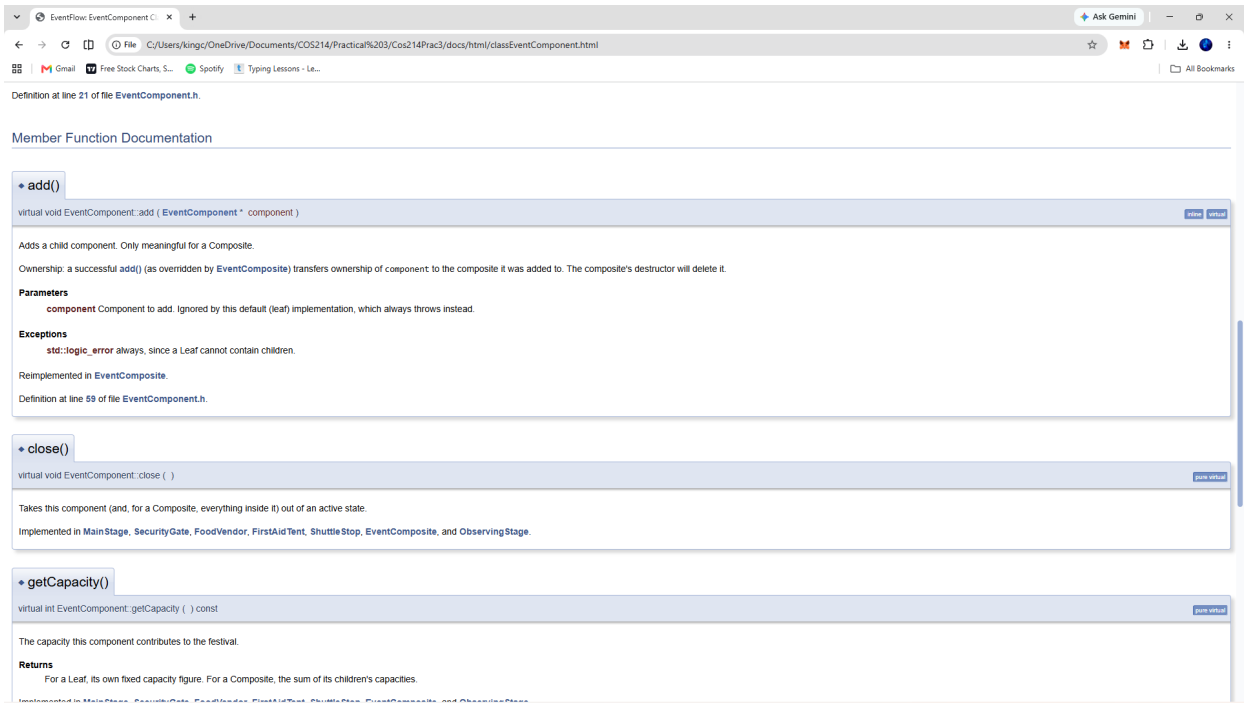
Shows the festival's signature reorganisation scenario across six lifelines: the Taco Stand is moved from Food Court Precinct to Main Stage Zone through the same four operations CascadingZone::transferChild() performs internally (remove, detach, add, attach), a festival-wide notice cascades through a loop fragment via each zone's real notify()/update() chain, and an emergency close() and a later observer detachment are also shown.



11. Doxygen Evidence

Running doxygen Doxyfile generates the documentation below in docs/html, with no warnings. The landing page and class list are shown here as evidence of successful generation.





12. GitHub Repository and Workflow Reflection

The team's private GitHub repository for this practical is:

https://github.com/Fluffyboyyy/Cos214Prac3

This section was prepared by Gary Mull (Task 7), drawing on the repository's commit and pull request history. The team divided the work along the practical's own task boundaries. Gary Mull built Task 2 (the Composite structure: EventComponent, EventComposite and the five concrete leaf classes) on a compositeEvent branch, and Task 3 (the Observer structure: Subject, Observer, EventNotice and CascadingZone) on an EventObserver branch, each merged into main through a pull request. Louwrens Johansen built Task 5 (the four UML sequence diagrams) on an SD branch, merged into main through a pull request. Chitambala Sikazwe worked on Task 1 (event concept and architecture), Task 4 (event rules and dynamic behaviour), Task 6 (Doxygen documentation) and Task 8 (integration and demonstration), committing directly to main, since this work built additively on top of the Composite and Observer branches once they had already been merged in, rather than needing its own review branch.

Changes were integrated through GitHub pull requests for the two large structural components and the sequence diagrams: compositeEvent was merged into main twice (pull requests #1 and #3) as the Composite implementation was completed and then extended, EventObserver was merged twice (pull requests #2 and #4) as the Observer implementation was completed and then its written theory answers were added, and SD was merged once (pull request #5) once all four sequence diagrams were ready together. Task-scoped additions, such as PDFs, testing files and the Doxygen output, were committed directly to main once the branches they depended on had already landed, since they did not need review on their own branch.

Three commits and pull requests in particular show meaningful collaboration rather than a single end-of-project upload:

- Pull requests #1 and #3 ("Merge pull request #1/#3 from Fluffyboyyy/compositeEvent") show the Composite structure being iterated on its own branch and merged in twice, giving the rest of the

team a stable, working base to build the Observer system and later the event rules on top of, instead of everyone working against a single shared branch at once.

- The "merge: integrate main into compositeEvent" commit shows the compositeEvent branch being brought back in sync with main partway through development, rather than left to diverge until the final merge, which kept that pull request small and easy to review instead of arriving as one large, conflicting change.
- Pull requests #2 and #4 ("Merge pull request #2/#4 from Fluffyboyyy/EventObserver") show the Observer, Subject, EventNotice and CascadingZone classes being merged into main as a complete, working unit before Task 4's event rules were started, so that work could be built directly on a tested Observer system rather than an in-progress one.

Branches and pull requests helped specifically because Task 2, Task 3 and Task 5 are each large, structurally significant deliverables that later tasks depend on directly. Isolating each on its own feature branch meant main only ever received a complete, working unit of that task rather than a partial or broken intermediate state. That mattered most for Chitambala's tasks (1, 4, 6 and 8), all of which build directly on the Composite and Observer classes: because compositeEvent and EventObserver were merged in before that work began, main was always in a buildable state to start from.

13. Team Contribution Statement

Chitambala Sikazwe (u25173384)

Responsible for Task 1, Task 4, Task 6 and Task 8.

- Task 1, Event Concept and Completed Architecture: defined the City Lights Festival concept, the five concrete leaf types and the Venue/Zone/Precinct grouping structure; produced the GoF Composite/Observer participant mapping table, the completed UML class diagram, and the design rationale covering ownership, the Observer/Subject dual role, and the push model.
- Task 4, Event Rules and Dynamic Behaviour: implemented the five leaf-specific event rules, SecurityGate's conditional severity-threshold decision, the CascadingZone::transferChild() runtime reorganisation, and three original features (a SecurityGate VIP fast-track lane, FirstAidTent backup power, and ShuttleStop emergency rerouting).
- Task 6, Doxygen Documentation: added class-level and per-operation Doxygen comments across the codebase, documented three non-obvious design decisions in detail (Observer's non-owning references, Subject::notify()'s snapshot-copy iteration, and the dynamic_cast in transferChild()), and configured the Doxyfile so doxygen Doxyfile generates the browsable documentation with no warnings.
- Task 8, Integration and Demonstration: built the single coherent end-to-end simulation (Task8Demo.cpp) covering every Task 8.1 requirement in one run, wrote the project's Makefile, and verified the full build under AddressSanitizer and UndefinedBehaviorSanitizer. During final verification, this testing surfaced a real bug in CascadingZone::update() (a blanket composite close() was overriding FirstAidTent's own decision to stay open, and it never recovered on the following OPEN notice); root-caused and fixed it by removing the redundant composite-level close() so every unit's reaction goes through its own update() only, then re-verified the fix under the full regression suite and sanitizers.

Gary Mull (u25071247)

Responsible for Task 2, Task 3 and Task 7, developed on the compositeEvent and EventObserver branches.

- Task 2, Composite Event Structure: implemented the EventComponent interface, the EventComposite class (owning children vector, recursive open()/close()/reportStatus()/getCapacity()), and the five concrete leaf classes (MainStage,

SecurityGate, FoodVendor, FirstAidTent, ShuttleStop), each with genuinely different behaviour rather than just a different printed name. Produced the Composite object diagram and the written explanation of destruction and ownership transfer (2.3, 2.4).

- Task 3, Observer Notification System: implemented the Subject and Observer base classes, EventNotice/NoticeType, and CascadingZone, the class that combines a Composite group with the Subject and Observer roles. Defined and documented the attach()/detach() registration policy (duplicate attach is a no-op, detaching an unregistered observer is safely ignored), the decision that the client is responsible for an observer's lifetime rather than Subject or Observer tracking it automatically, and the push-model justification for Subject::notify() (3.1, 3.2, 3.5).
- Task 7, Git and GitHub Engineering Workflow: prepared the workflow reflection in section 12 above, using the repository's own commit and pull request history to document how the team divided and integrated its work.

Louwrens Johansen (u25607414)

Responsible for Task 5, developed on the SD branch.

- Task 5, Sequence Diagram Portfolio: produced the four UML sequence diagrams (SD1 to SD4). SD1 shows the client building the Composite tree and establishing Observer registrations, distinguishing ownership from registration. SD2 shows a notice cascading through three runtime levels using a loop and an alt fragment. SD3 models a conditional event response at River Side Zone, and SD4 is the team's signature scenario, covering a runtime reorganisation, a festival-wide cascading notice, and an emergency shutdown across six lifelines. During final integration, a code fix to CascadingZone::update() (see Chitambala's Task 8 note above) meant SD3 and SD4's affected fragments were revised to keep matching the corrected behaviour exactly; the overall diagrams, scenarios and lifeline structure are Louwrens's original work.

How the team coordinated its work

The team divided work along the practical's own task boundaries and coordinated through GitHub. Task 2 and Task 3 were each developed on a dedicated feature branch and merged into main through a pull request; Task 5 was developed on its own branch and merged the same way. Tasks 1, 4, 6 and 8 were committed directly to main, since that work built additively on top of the Composite and Observer branches once they had already been merged in. Further detail on the team's GitHub workflow, including specific commits and pull requests that show this coordination, is in section 12 above.

14. Additional Written Answers

14.1 Task 8 demonstration route (about five minutes)

main.cpp runs every demonstration unconditionally: the individual Task 2, Task 3 and Task 4 demonstrations, followed by runEventFlowDemo() in Task8Demo.cpp, the single continuous simulation described below. Running everything on every invocation, rather than gating the earlier demonstrations behind a command-line flag, means a plain run of the built executable with no arguments exercises every task's code path in one pass, which matters for automated tools (such as an autograder measuring code coverage) that simply execute the program directly. Each requirement below is covered and printed to the console as a clearly labelled step, so a marker can follow along without reading the code.

Requirement (Task 8.1)	**Where it happens**
Construction of the Composite	STEP 1: a root Venue, two Zones, one nested Precinct, five leaf types.
Observer registration	STEP 2: every zone and leaf attach()ed, kept separate from Composite ownership.

Requirement (Task 8.1)	Where it happens
A Composite traversal/query	STEP 3: reportStatus() and getCapacity() recurse through the whole tree before anything happens.
At least three different notices	STEPS 4, 6, 8, 9: SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT, EVACUATE and OPEN (five in total).
At least one cascading notification	Every issueNotice() call cascades; STEP 6 specifically reaches three runtime levels (root, River Side Zone, Food Court Precinct, Taco Stand).
A registration change	STEP 5: North Gate is detach()ed, a notice is sent and confirmed not to reach it, then it is re-attach()ed and reacts normally again.
A runtime reorganisation	STEP 7: transferChild() moves the Taco Stand from Food Court Precinct to Main Stage Zone, ownership and observer registration both move.
Clean shutdown	STEP 10: a single delete on the root, verified leak-free and dangling-pointer-free under AddressSanitizer/UndefinedBehaviorSanitizer.

For the live demonstration, the relevant five minutes are the final ten steps (Task8Demo.cpp's runEventFlowDemo()), printed after the Task 2, Task 3 and Task 4 demonstrations have already run. The ten printed steps map directly onto a five minute walkthrough: STEP 1 to 3 establish the Composite tree and a baseline query; STEP 4 shows a notice reaching every leaf even where most have no reaction, itself a deliberate difference between leaf types; STEP 5 is the registration change, showing the practical's own requirement that observers be attached, detached and notified at runtime; STEP 6 is the weather alert, showing five different leaf reactions and a notice reaching a leaf three runtime levels below the root; STEP 7 is the reorganisation, showing both the Taco Stand's ownership and its observer registration move together; STEP 8 to 9 are the evacuation and recovery; STEP 10 is the clean shutdown.

14.2 The Makefile

No task explicitly owned the Makefile the brief requires (build with make, producing an executable named eventflow), so it was added as part of integration. It compiles main.cpp, Task2Testing.cpp, Task3Testing.cpp, Task4Testing.cpp, Task8Demo.cpp, Subject.cpp, CascadingZone.cpp and EventComposite.cpp, and links them into eventflow. EventComponent.cpp is deliberately not compiled as its own translation unit: it defines the five concrete leaf classes and every test/demo file that needs them includes it directly, so it is compiled once per file that includes it rather than separately.

```
make        # builds eventflow
```